Clase 13 — Correo electrónico: SMTP, POP3, IMAP y MIME

Bloque: B (Aplicación) · **Semana:** 5 · **Duración estimada:** 110 min **Correlativa:** Clase 12 (taller DNS + HTTP) · **Sigue con:** Clase 14 (seguridad en la aplicación: TLS, autenticación y correo seguro) **Material de referencia:** Tanenbaum cap. 7 (§7.1) · Exámenes P2, P4, Ejercicio 2

> ⚠️ **Sobre los números de RFC de esta clase:** , que es la fuente de verdad del curso para números de RFC, **no tiene una sección de correo electrónico**. Los RFC que se citan acá son los oficiales y están verificados contra (octubre de 2026), pero **verificalos vos** en antes de copiarlos en un examen. Los **puertos**, en cambio, sí están verificados contra el registro de IANA en §15.

🎯 Al terminar esta clase vas a poder

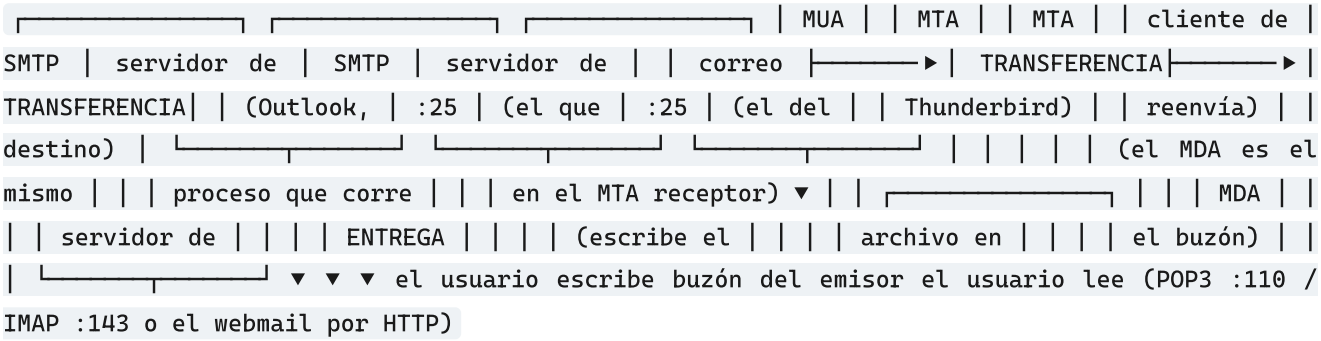
1. Dibujar la **arquitectura del correo** con sus **tres roles** (MUA, MTA y MDA) y explicar qué protocolo usa cada par de actores.
2. Explicar por qué SMTP es un protocolo **push** y por qué POP3 e IMAP son **pull**, y qué consecuencia de seguridad tiene eso.
3. Escribir la **transcripción literal** de una sesión SMTP completa, con `EHLO`, `MAIL FROM`, `RCPT TO`, `DATA` y el punto final, incluida una respuesta de error.
4. Explicar **por qué existe MIME** y aplicar los tres (, ,) sobre **el mismo texto**.
5. Describir los **comandos y los dos estados** de POP3, y explicar por qué sus comandos van en claro.
6. Comparar POP3 e IMAP en una tabla de **8 filas**, entendiendo la diferencia de fondo: **dónde vive el estado**.
7. Resolver completo el **Examen 2010-03 P2** (8 puntos) y el **Examen 2008-12 P4** (7 puntos), con las soluciones oficiales a la vista.

> **Prerequisito del examen:** (8 puntos — ★ la arquitectura del correo, los roles y los protocolos), (7 puntos — ★ el de POP3 y el *dot-stuffing*), Ejercicio 2 (el diálogo SMTP transcrito y el *dot-stuffing* en la práctica), (firma digital aplicada al correo).

PARTE 1 — TEORÍA

1.1 La arquitectura del correo: tres roles y un camino largo

El correo electrónico es la aplicación más vieja de Internet y la única que sigue siendo **extremo a extremo**. Esa decisión de diseño explica toda la arquitectura.



Los tres roles, con su glosa y su pregunta

Rol	Nombre en inglés	Glosa	La pregunta que responde	Protocolo hacia el siguiente
MUA	*Mail User Agent*	agente de usuario de correo	«¿qué escribo y qué leo?»	SMTP (push) hacia el MTA propio
MTA	*Mail Transfer Agent*	agente de transferencia	«¿a quién se lo mando y cómo lo hago llegar?»	SMTP (push) hacia otro MTA
MDA	*Mail Delivery Agent*	agente de entrega	«¿dónde lo archivo para que el destinatario lo encuentre?»	POP3 o IMAP (pull), o HTTP

El nombre importa porque cada nombre dice una cosa:

- **User**: el que **usa** el correo. Es el único que puede escribir y leer.
- **Transfer**: el que lo **traslada**. No lee nada, no guarda nada: reenvía.
- **Delivery**: el que lo **deja en su lugar**. Típicamente corre **dentro del mismo proceso que el MTA receptor**, pero conceptualmente es otro rol.

Por qué el camino es largo, y por qué eso es una decisión y no un accidente

` El correo NO es extremo a extremo en la entrega. El emisor no habla con el receptor: habla con SU servidor, y ese servidor habla con el del receptor. Hay al menos un punto —un servidor que no es el emisor— que puede leer o modificar el mensaje.

¿Por qué? Porque en 1982 no existía la infraestructura para que un usuario cualquiera aceptara conexiones entrantes. El teléfono de la época no tenía fija: no había forma de saber a qué máquina llamar. El correo se diseñó como la **bandeja**: el emisor deja el mensaje en un buzón y el destinatario, cuando quiere, pasa a recogerlo.

TODAY, con IP, esa razón ya no existe, y por eso la bandeja es un LASTRE: el correo sigue siendo el único protocolo de aplicación que no es extremo a extremo. `

> **La frase del examen:** *el correo electrónico es un sistema de_STORE-and-forward (guardar y reenviar)extremo a extremo.* Guardar y reenviar es lo que hace el MTA; extremo a extremo es lo que le falta.

El rol que falta en el diagrama y que hay que dibujar igual: el **MUA del receptor**

` MUA ──SMTP:25──▶ MTA ──SMTP:25──▶ MTA/MDA ──POP3:110──▶ MUA escribir (reenvía) (deposita) leer

El MUA del receptor entra por POP3, IMAP (o por HTTP si es webmail). Y si usa IMAP, en realidad hay un cuarto rol en juego: el estado (bandejas,Flags, carpetas) vive en el servidor, no en el cliente. `

Y un quinto rol que aparece cuando hay varios servidores de correo en el mismo dominio: el **MTA de reenvío** o *forwarder*, que recibe el mensaje y lo pasa a otro MTA sin depositarlo. Es el que existe cuando decís «redireccioná el correo a otra dirección».

1.2 SMTP: el protocolo push del envío

SMTP (*Simple Mail Transfer Protocol*) tiene el nombre de «simple» porque lo más importante es que hace una sola cosa: **empujar un mensaje**. No pide, no busca, no responde preguntas de directorio. Solo empuja.

Dato	Valor
Puerto principal	**25 TCP** (registrado también en UDP, pero el uso real es TCP)
Puerto de envío del cliente	**587** (*submission*), con autenticación
Puerto TLS implícito	**465** (*submissions*), deprecated a favor de en 587
Transporte	TCP, **sin cifrar** en el puerto 25
RFC	(2008, *Internet Standard*), que reemplaza a (1982) · *verificar*
Formato del mensaje	(2008), que reemplaza a (1982) · *verificar*

Los dos modos de usar SMTP, y por qué existen los dos puertos

` 25 (mail server → mail server) "MTA a MTA":Relay, sin autenticación, porque los dos extremos son servidores conocidos entre sí.

587 (cliente → su mail server) Submission: el usuario se identifica con usuario y contraseña, porque si no, cualquiera podría mandar correo en tu nombre.

465 (submission con TLS implícito) Todo el canal va cifrado desde el principio. Deprecada a favor de 587 con STARTTLS, pero sigue registrada y usada. `

> **Por qué importa en el examen:** SMTP **no tiene autenticación por defecto**. Por eso hay dos puertos: el 25 para el salto servidor a servidor y el 587 para el salto cliente a servidor. El 587 no existe por otra cosa.

El diálogo, comando por comando

```
El cliente (MUA) El servidor (MTA) | | |◄──────── 220 <dominio> ESMTP Service ready ────────| (saludo) | | |──────── EHLO <mi.dominio> ──────────────────────────►| (o HELO, antiguo) |◄── 250-<dominio> (saludo y capacidades) ────| | 250-SIZE 35882577 | | 250-8BITMIME | | 250-STARTTLS | | 250-AUTH PLAIN LOGIN | | 250 HELP | | | |──────── MAIL FROM: <origen@dominio.com.uy> ──────────►| (1 solo por mensaje) |◄──────── 250 2.1.0 Ok ──────────────────────────────| | | |──────── RCPT TO:<destino@dominio.com.uy> ──────────►| (1 o varios) |◄──────── 250 2.1.5 Ok ──────────────────────────────| | | |──────── DATA ──────────────────────────────────────►| |◄──────── 354 Start mail input; end with <CRLF>. <CRLF> —| | | |──────── (el mensaje, línea por línea) ──────────► | |──────── . ──────────────────────────────────────►| (punto solo) |◄──────── 250 2.0.0 Ok: queued as 40AB06AF005B39F9 —| | | |──────── QUIT ──────────────────────────────────► | |◄──────── 221 <dominio> closing connection ────────|
```

Los cinco comandos que hay que saber de memoria:

Comando	Qué hace	Cuántas veces
(o)	saludo e identificación; además lista las capacidades	1, al abrir
	declara el **remitente** (la casilla del *return path*)	**1 sola** por mensaje
	declara un **destinatario**	**1 o varias**
	«ahora viene el mensaje»	1, antes del cuerpo
	cierra la sesión	1, al terminar

Y las dos reglas del punto que hacen que todo funcione:

` 1. El mensaje se termina con una línea que contiene UN SOLO PUNTO: . Ese punto no es parte del mensaje: es el delimitador.

2. Si una línea del mensaje empieza con un punto, el emisor la "duplica": .algo en el texto → ..algo en el cable El receptor, al leer, saca un punto de cada línea que empiece con punto.

Esto se llama DOT-STUFFING y lo vas a tener que explicar en el examen. `

La transcripción literal de un envío completo

Esta es la forma que se ve en un o en Wireshark, con las líneas del servidor () y del cliente ():

```
S: 220 mta4.in.adinet.com.uy ESMTP Service (7.0.027) ready C: EHLO cliente.fing.edu.uy S: 250-mta4.in.adinet.com.uy S: 250-SIZE 35882577 S: 250-8BITMIME S: 250-STARTTLS S: 250 HELP C: MAIL FROM:<origen@dominio.com.uy> S: 250 2.1.0 Ok C: RCPT TO:<dest1@dominio.com.uy> S: 250 2.1.5 Ok C: RCPT TO:<dest2@dominio.com.uy> S: 550 5.1.1 <dest2@dominio.com.uy>: Recipient
```

`address rejected: User unknown C: RCPT TO:<dest3@dominio.com.uy> S: 250 2.1.5 Ok C: DATA S: 354 Start mail input; end with <CRLF>.<CRLF> C: From: Ana Perez <origen@dominio.com.uy> C: To: Destino Tres <dest3@dominio.com.uy> C: Subject: Hola a todos C: Date: Thu, 12 Mar 2026 10:14:07 -0300 C: C: Hola: C: C: .Este renglon empieza con un punto, asi que viaja duplicado. C: ....Y este renglon tambien. C: Saludos C: . S: 250 2.0.0 Ok: queued as 40AB06AF005B39F9 C: QUIT S: 221 2.0.0 mta4.in.adinet.com.uy closing connection`

Leé los tres detalles que hacen la diferencia:

1. ** puede fallar sin que falle el mensaje. **El dio (*user unknown*), y el envío** siguió adelante **con `DATA`. Por eso `QUIT` es el comando que se repite:** un mensaje puede tener varios destinatarios y algunos pueden fallar**.
2. **El `DELE` es permanente y el `EXPUNGE` es temporal.** `+OK` = no reintentar, el usuario no existe. `-ERR` = Mailbox busy, reintentá más tarde. Esa distinción es la que permite que un MTA no tire a la basura un correo por un fallo pasajero.
3. **Las dos líneas con puntos duplicados.** El mensaje original tenía dos puntos al principio de la línea, y viajan como `..` y `.` . El punto final de la transacción es un punto **solo**.

Los códigos de respuesta que hay que reconocer

`2xx ÉXITO 250 Ok · 220 Service ready · 354 Mandame los datos 4xx ERROR TEMPORAL Intentá más tarde. La transacción NO se pierde: el MTA guarda el mensaje y reintenta (cola). 5xx ERROR PERMANENTE No hay caso, no reintentes. El MTA genera un reporte de-states (bounce) para avisarle al emisor.`

> **La frase del examen:** *SMTP es un protocolo de códigos de respuesta: el cliente interpreta un número de tres dígitos y decide si sigue, espera o abandona.*

1.3 MIME: por qué existe y cómo funciona

El problema que vino a resolver

` RFC 822 (1982) decía: un mensaje es texto ASCII de 7 bits, con líneas de 1 000 caracteres como máximo.

Eso alcanza para escribir una carta. No alcanza para nada de lo que la gente se empezó a mandar enseguida:

• tildes y eñes → no son ASCII de 7 bits • fotos, PDF, planillas → son bytes binarios, y un binario puede tener un "cambio de línea" donde sea •firmas → imágenes que garbage en el medio `

Y acá está el error de diseño de fondo: SMTP transporta **líneas terminadas en** . Un archivo binario puede contener la secuencia de bytes en medio de los datos. Si eso viaja sin escapar, **el protocolo cree que ahí terminó una línea y que la siguiente es un comando nuevo**. El archivo se rompe.

Las dos herramientas de MIME

` MIME = Multipurpose Internet Mail Extensions (RFC 2045 a 2049, 1996) · verificar

HERRAMIENTA 1 — Content-Type: ¿QUÉ es esto? Dice el TIPO de contenido: texto, imagen, PDF... Y permite meter un ARCHIVO DENTRO del mensaje: multipart/*

HERRAMIENTA 2 — Content-Transfer-Encoding: ¿CÓMO viaja? Dice cómo se codificó el cuerpo para que no rompa el transporte: 7bit · quoted-printable · base64 `

: los tres que hay que saber

Valor	Qué hace	Cuándo se usa	Costo
****	no toca nada: el cuerpo ya es ASCII de 7 bits	texto sin acentos ni caracteres raros	0 %
**** (QP)	escapa lo que no es ASCII con en hexadecimal, y **deja intactos los espacios y los acentos imprimibles**	texto con alguna tilde, o líneas largas	~10-20 %
****	codifica **todo** en grupos de 6 bits → caracteres imprimibles, con de relleno	**cualquier binario**: imágenes, PDF, adjuntos	**+33 %** (4 caracteres por cada 3 bytes)

La regla de oro para el examen:

> **`base64` es para binario; `quoted-printable` es para texto casi legible.** Un texto con tildes va en `quoted-printable`, nunca en `base64`: es legible y pesa menos.

: los tipos que hay que reconocer

` text/plain texto sin formato text/html una página web text/html; charset=UTF-8 lo mismo, dizendo la codificación image/png una imagen PNG application/pdf un PDF application/octet-stream "cualquier binario", el comodín

Y los dos compuestos, que son la parte interesante:

multipart/alternative "acá hay varias VERSIONES del mismo mensaje, elegí la mejor" → texto plano + HTML

multipart/mixed "acá hay varios CONTENIDOS distintos que van juntos" → texto + un adjunto `

La diferencia entre POP3 e IMAP, que es el punto que más se confunde:

` multipart/alternative El cliente ELIGE UNO de los que van dentro. Típico: texto plano (1er lugar, el más simple) + HTML (2.º, el más rico). El orden importa: el primero es el menosPreferred.

multipart/mixed El cliente los puede ver TODOS, y no elige. Típico: el texto del correo + un PDF adjunto.

Y se pueden anidar: multipart/mixed ├── text/plain (el texto del correo) └── application/pdf (el adjunto, en base64) `

Ejemplo con los tres encodings sobre el **mismo texto**

Texto original:

`Hola Ana: El precio es 1500 $ (dólares) y la entrega es en 3 días. Te adjunto el presupuesto. Saludos, Pedro`

Con : no se puede. La línea tiene caracteres de 8 bits, y **prohíbe** exactamente eso. Hay que elegir otro encoding.

Con :

` Content-Type: text/plain; charset=UTF-8 Content-Transfer-Encoding: quoted-printable

Hola Ana: El precio es 1500 $ (d=F3lares) y la entrega es en 3 d=EDas. Te adjunto el presupuesto. Saludos, Pedro `

`Fijate: "ó" (0xF3) se volvió "=F3" y "í" (0xED) se volvió "=ED". Los espacios, los signos y los números quedaron IGUALES: el texto sigue siendo legible por un humano. Eso es lo que hace que QP sea el encoding correcto para texto con caracteres no ASCII.`

Con :

` Content-Type: text/plain; charset=UTF-8 Content-Transfer-Encoding: base64

SG9sYSBBbmE6CkVsIHByZWNpbyBlcyAxNTAwICQgKGRvbGFyZXMpIHkgbGEg
bGEgZGVsaXZlcmEgZXMgZW4gMyBkw6lhcy4KVGUgYWRqdW50byBlbCBwcmVz
dWVzdG8uIFNhbHVkb3MsIFBlZHJv `

` QEIgc29zIHRlbmllbW8gdGllbmUgZXMgZXMgNTUzIGJ5dGVzLg==

El texto SIEMPRE queda ilegible, porque base64 no distingue un texto de un binario: para él son solo bytes. Por eso, si lo que mando es texto, base64 es la peor elección posible. `

El ejemplo completo: un correo con adjunto ()

` From: Pedro Gomez pedro@dominio.com.uy To: Ana Perez ana@otraempresa.com.uy Subject: Presupuesto Date: Thu, 12 Mar 2026 10:14:07 -0300 MIME-Version: 1.0 Content-Type: multipart/mixed; boundary="======frontera123"

--======frontera123 Content-Type: text/plain; charset=UTF-8 Content-Transfer-Encoding: quoted-printable

Hola Ana:

Te adjunto el presupuesto. La entrega tarda 3 d=EDas.

Saludos, Pedro

--======frontera123 Content-Type: application/pdf; name="presupuesto.pdf" Content-Transfer-Encoding: base64 Content-Disposition: attachment; filename="presupuesto.pdf"

JVBERi0xLjQKJcOkw7zDtsOfCjIgMCBvYmoKPDwvTGVuZ3RoIDMgMCBSL0ZpbHRlci9GbGF0
ZURlY29kZT4+CnN0cmVhbQp4nDPQM1Qo5ypUMABCA==

--======frontera123-- `

Las cinco cosas que hay que mirar en ese mensaje:

1. **El ** es una cadena que** no existe dentro del mensaje** y que se usa para separar las partes. Es la clave del formato MIME.
2. **Cada parte es un mini-mensaje**, con sus **propias cabeceras MIME** (y).
3. **El PDF va en** , porque es binario. Un PDF en no solo sería ilegible: se rompería.
4. **** le dice al cliente «esto es un adjunto, no lo muestres en el cuerpo».
5. **El delimitador final tiene dos guiones menos** (), que es lo que marca el cierre de la parte *multipart*.

1.4 POP3: el protocolo pull de la lectura

POP3 (*Post Office Protocol version 3*) hace **una sola cosa**: descargar mensajes del buzón al cliente y, opcionalmente, borrarlos del servidor.

Dato	Valor
Puerto	**110 TCP** (seguro: **995**, *pop3s*, con TLS implícito)
Modelo	**pull**: el cliente va a buscar
Estado	en el **cliente**, después de descargar
RFC	(1996) · *verificar*
Transporte	TCP, **en claro** en el 110

Los dos estados, y por qué existen

```
┌──────────────────────────┐
│  ESTADO DE AUTORIZACIÓN  │  Fase 1: el servidor solo admite comandos
└──────────────────────────┘  de autenticación y de info del buzón.
        Todo lo demás da -ERR.
   USER  PASS  (+ APOP, opcional)
                  │
                  ▼
┌──────────────────────────┐
│  ESTADO DE TRANSACCIÓN   │  Fase 2: acá están los comandos que mueven
└──────────────────────────┘  mensajes. Y acá hay una subtlety:
   hasta que el cliente manda QUIT, el servidor NO borra nada, aunque el
   cliente haya mandado DELE.
                  ▼
            QUIT ──►  se borran los marcados con DELE
```

La subtlety es lo más importante del protocolo y casi nunca se dice:

> ** no borra nada al instante: marca. **El servidor borra de verdad** solo cuando el cliente manda `QUIT`. Si la conexión se cae antes del `QUIT`, los mensajes marcados **se quedan en el buzón**. Es un mecanismo de seguridad contra la pérdida de datos por un corte de conexión, y por eso el estado «de actualización» se llama así.

Los comandos

Comando	Qué hace
	envía el nombre de usuario
	envía la contraseña
	devuelve **dos números**: cantidad de mensajes y tamaño total en bytes
	devuelve los números de los mensajes y sus tamaños
	descarga el mensaje **completo**, con sus cabeceras
	marca el mensaje para borrar
	desmarca todo
	cierra la sesión y **aplica los borrados**

Una sesión POP3 completa, literal

```
C: USER ana.perez S: +OK POP3 server ready C: PASS ****** S: +OK maildrop has 3 messages C: STAT S: +OK 3 120458 C: LIST S: +OK 3 messages (120458 octets) S: 1 48210 S: 2 40122 S: 3 32126 C: RETR 1 S: +OK 48210 octets S: From: pedro@dominio.com.uy S: To: ana.perez@otraempresa.com.uy S: Subject: Presupuesto S: S: Hola Ana: S: S: Te adjunto el presupuesto. S: . C: DELE 1 S: +OK message 1 deleted C: QUIT S: +OK POP3 server signing off (21600 seconds)
```

Tres cosas que hay que notar:

1. ** es éxito y `-ERR` es error. **Todos los comandos empiezan por `+OK` o `-ERR`, y el** código numérico va después**, para las respuestas multi-línea.** SMTP usa un esquema de tres dígitos (,) y POP3 usa dos tipos de línea** (,). Es una diferencia de diseño entre los dos protocolos que se pregunta.
2. ** devuelve el mensaje entero, cabeceras incluidas.** No hay forma de pedir «solo el cuerpo» ni «solo las cabeceras».
3. **`DELE` dice `+OK`, pero todavía no se borró.** Se borró en el `QUIT`.

Por qué los comandos van en claro

```
┌──────────────────────────────────────────────────────────────┐ │ POP3 y SMTP en los puertos 110 y 25 son TEXTO PLANO. │ │ No hay cifrado. │ │ │ Eso significa que CUALQUIERA en el camino —cada router, cada │ │ servidor de la red local, cualquier espía— puede leer: │ │ │ │ USER ana.perez ← tu nombre de usuario │ │ PASS MiContraseñaSecreta ← tu contraseña, EN CLARO │ │ RETR 1 ← y el mensaje entero │ │ │ Por eso existen los puertos seguros: │ │ 995 = POP3 sobre TLS desde el primer byte (pop3s) │ │ STARTTLS en el 110: se cifra a mitad de camino │ └──────────────────────────────────────────────────────────────┘
```

> **La respuesta corta para el examen:** *los comandos van en claro porque POP3 fue diseñado en 1984, cuando el cifrado en la capa de aplicación no existía y la seguridad se resolvía con contraseñas y con no*

> *dejar pasar la metallurgy. La respuesta larga: POP3 no tiene cifrado propio, no lo tiene por diseño, y por eso el puerto seguro es otro puerto distinto (995) y no una opción del protocolo.*

1.5 IMAP: el mismo pull, pero con el estado en el servidor

IMAP (*Internet Message Access Protocol*) también es **pull**, pero tiene una idea central que POP3 no tiene: **la sesión es una vista de una estructura de carpetas que vive en el servidor**.

Dato	Valor
Puerto	**143 TCP** (seguro: **993**, *imaps*)
Modelo	**pull**, pero **el estado queda en el servidor**
RFC	(2001) *obsoleta* por (2021), *IMAP4rev2 · verificar*
Transporte	TCP; en el 143, TLS implícito en el 993

Los comandos que hacen la diferencia

` IMAP tiene los mismos DOS estados que POP3 (no autenticado / autenticado), pero los comandos de la fase autenticada son de otro tipo:

SELECT INBOX "abrí el buzón de entrada" ← POP3 NO TIENE NADA DE ESTO EXAMINE INBOX lo mismo, pero en modo solo lectura LIST "" * "dame la lista de carpetas" ← POP3 NO TIENE ESTO SEARCH UNSEEN "buscá lo que no leí" ← POP3 NO TIENE ESTO FETCH 1 BODY.PEEK[] "decime el mensaje, pero no me lo borre" STORE 1 +FLAGS (\Seen) "marcalo como leído, en el SERVIDOR" APPEND INBOX {n} "depositá un mensaje en el servidor" UID EXPUNGE "borrá lo que marqué, en el SERVIDOR" IDLE "quedate esperando que te avise si llega algo" `

** es la razón por la que IMAP «se siente» moderno: **el servidor puede** empujar una notificación* *(«te llegó un correo») aunque IMAP sea un protocolo* pull*. El cliente sigue preguntando, pero puede esperar sin vueltas.

La tabla comparativa, que es lo que hay que saber de memoria

	POP3	**IMAP**
Modelo	pull	pull (con ,:notificación)
Puertos	110 / 995	143 / 993
¿Dónde queda el estado?	en el **cliente**	en el **servidor**
¿Servidor?	¿borra después de descargar? (opcional)	no: el mensaje sigue en el servidor
¿Carpetas?	**no tiene** (una sola «bandeja»)	**sí**: , , , , y las que crees vos
¿Estado de lectura?	local del cliente	**flags en el servidor** (, ,)
¿Busca en el servidor?	no: baja todo y busca local	**sí**: , y el servidor solo manda lo que le pedís
Ideal para	una casilla, un dispositivo, una línea	varios dispositivos, correo grande, carpetas

> **La frase del examen:** *la diferencia entre POP3 e IMAP no es de transporte ni de comandos: es dónde vive el estado. POP3 es un buzón plano que vos venís a vaciar; IMAP es un sistema de archivos remoto con carpetas y marcas, y vos venís a mirar.*

La diferencia práctica, en un ejemplo

` El mismo usuario, en el celular y en la notebook, con 4 000 correos:

CON POP3: En la notebook, POP3 baja los 4 000 y los BORRA del servidor. En el celular: "no hay correos". Y si los baja y los borra también, la notebook dice: "no hay correos". → POP3 es de UNO en uno, o tenés que deixar de borrar.

CON IMAP: En la notebook, IMAP deja los 4 000 en el servidor y marca los leídos (flag \Seen) ahí mismo. En el celular: ve los mismos 4 000, en las mismas carpetas, con los mismos flags. "Leíste esto en la notebook" también lo sabés en el celular. → IMAP es de MUCHOS en muchos. `

1.6 ⚠️ Actualización a 2026

Sigue igual — de memoria, sin dudar:

- Los **tres roles**: MUA, MTA y MDA, y la cadena **MUA → MTA → MDA → MUA**.
- **SMTP = push**, puerto **25**, con / / / / el punto final / , y los códigos de tres dígitos (, ,).
- ** se repite** y puede fallar sin tumbar el mensaje.
- **POP3 = pull**, puerto **110** (seguro **995**), con los dos estados y con que solo borra en el `QUIT`.
- **IMAP = pull con estado en el servidor**, puerto **143** (seguro **993**), con carpetas y *flags*.
- **MIME** existe porque RFC 822 era ASCII de 7 bits; los tres encodings; `multipart/alternativemultipart/mixed`.

Cambió, y es lo que hay que contar en 2026:

- ** (2021, IMAP4rev2) reemplaza a .* *Mismo protocolo, especificación reescrita: se eliminaron comandos obsoletos y se agregó soporte de UTF-8.* Verificar el número antes de citarlo.*
- **El correo sin cifrar es historia, no práctica.** Los puertos 25, 110 y 143 **deben** terminar en `QUIT`; los puertos **465**, **993** y **995** son de TLS implícitocito. Es la diferencia con la clase de 2011, donde el curso mostraba sesiones en claro.
- **La autenticación del envío es obligatoria en la práctica.** (2007) define y hoy todos los grandes proveedores la exigen en el 587. *Verificar.*
- **El antispam se movió de capa aplicación a capa red:** (un de DNS), (una firma criptográfica en la cabecera) y (qué hacer si los dos fallan) son **obligatorios** para que un mensaje llegue a la bandeja de entrada. Consecuencia conceptual: **hoy un correo electrónico sin SPF ni DKIM es indistinguible de un impostor**, y por eso el modelo sin cifrar ya no es sustainable.
- **El spam le ganó a la arquitectura.** LaFilterClash es un problema de aplicación que hoy se resuelve con reputación de IP y de dominio (), listas grises y reputación de remitente —o sea, con info que **vive fuera del protocolo SMTP**.

- **El «webmail» se comió el cliente**: hoy lo más usado es leer el correo por **HTTPS** en un navegador, que es una cuarta forma de acceder al buzón que el curso de 2011 mencionaba al pasar.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 (guía) — Un envío SMTP con adjunto, transcrito

Datos. Ana envía un correo a Pedro con un PDF adjunto. El mensaje de Ana tiene acentos, así que el texto va en `quoted-printable`; el PDF es binario y va en `base64`.

a) ¿Qué puerto usa y por qué?

` Puerto 587 (submission), NO el 25.

El 25 es para el salto MTA → MTA, donde los dos extremos son servidores conocidos. Ana → servidor de correo es un salto de USUARIO a servidor, y ahí hace falta autenticación. El 587 es exactamente ese caso. `

b) La transcripción completa del envío:

```
S: 220 mail.uy.com.uy ESMTP Service ready C: EHLO notebook.ana.uy S: 250-mail.uy.com.uy S: 250-SIZE 10240000 S: 250-8BITMIME S: 250-STARTTLS S: 250-AUTH PLAIN LOGIN C: STARTTLS S: 220 2.0.0 Ready to start TLS ··· (aca empieza el cifrado: todo lo que sigue viaja dentro de TLS) ··· C: EHLO notebook.ana.uy S: 250-mail.uy.com.uy S: 250-AUTH PLAIN LOGIN C: AUTH PLAIN AGFuYS5wZXJlejpDb250cmFz+WE6 S: 235 2.7.0 Authentication successful C: MAIL FROM: <ana.perez@uy.com.uy> S: 250 2.1.0 Ok C: RCPT TO:<pedro.gomez@otraempresa.com.uy> S: 250 2.1.5 Ok C: DATA S: 354 End data with <CR><LF>.<CR><LF> C: From: Ana Pérez <ana.perez@uy.com.uy> C: To: Pedro Gómez <pedro.gomez@otraempresa.com.uy> C: Subject: Presupuesto C: Date: Thu, 12 Mar 2026 10:14:07 -0300 C: MIME-Version: 1.0 C: Content-Type: multipart/mixed; boundary="BOUND42" C: C: --BOUND42 C: Content-Type: text/plain; charset=UTF-8 C: Content-Transfer-Encoding: quoted-printable C: C: Hola Pedro: C: C: Te adjunto el presupuesto. La entrega tarda 3 d=EDas. C: Saludos, Ana C: C: --BOUND42 C: Content-Type: application/pdf; name="presupuesto.pdf" C: Content-Transfer-Encoding: base64 C: Content-Disposition: attachment; filename="presupuesto.pdf" C: C: JVBERi0xLjQKJcOkw7zDtsOfCjIgMCBvYmoKPDwvTGVuZ3RoIDMgMCBSL0ZpbHRl C: ci9GbGF0ZURlY29kZT4+CnN0cmVhbQp4nDPQM1Qo5ypUMABCA== C: C: --BOUND42-- C: . S: 250 2.0.0 Ok: queued as 8B7A0C1199E4F002 C: QUIT S: 221 2.0.0 mail.uy.com.uy closing connection
```

c) Las seis decisiones que hay que justificar:

#	Decisión	Por qué
1	Puerto **587**, no 25	es un salto de usuario a servidor: requiere autenticación
2	** dos veces**, no	el devuelve las capacidades; el es el comando antiguo de 1982
3	** antes de **	si mandás la contraseña sin cifrar, viaja en claro por la red
4	**base64**, no	manda de una; va en dos pasos y es más fácil de interceptar
5	Texto en `binary`	tiene un 0, QP lo deja legible; `base64` lo haría ilegible y 33 % más grande
6	PDF en ****	es binario, y si viaja crudo puede traer que rompe el protocolo

d) ¿Cuántos bytes de másîne el ?

` El PDF ocupa 3 000 bytes reales. base64 mete 4 caracteres por cada 3 bytes: 3 000 / 3 = 1 000 grupos 1 000 grupos × 4 caracteres = 4 000 caracteres Como además base64 mete un salto de línea cada ~76 caracteres, hay que sumar los retornos de línea:

4 000 / 76 = 52,6 → 53 saltos de línea

Total en el cable: 4 000 + 53 = 4 053 bytes Costo: 4 053 − 3 000 = 1 053 bytes = 35 % de aumento `

Interpretación física: un adjunto de 10 MB llega por SMTP con unos **13,5 MB**. Y todo eso pasa **una sola vez por cada servidor de la cadena**, que es una de las razones históricas por las que el correo con adjuntos grandes «iba tan lento».

2.2 Ejercicio 2 — Los tres encodings sobre el mismo texto, comparados

Datos. El texto es

Encoding	Cómo queda en el cable	Legible	Tamaño
****	**no se puede usar**: el es 0xF3, de 8 bits	—	—
****		**sí**	32 bytes (el pasó de 1 a 3)
****		no	48 bytes (4 caracteres por cada 3 bytes)

La comparación que hay que saber de memoria:

` El ORIGINAL tiene 29 bytes y un solo carácter problemático: el "ó".

QP : 29 − 1 + 3 = 32 bytes. Solo paga el problema. base64: 29 bytes → 40 de datos + 4 de relleno + los saltos de línea = 48 bytes. Paga TODO, incluso la parte que ya era ASCII.

QP gana en legibilidad y en tamaño SIEMPRE que haya texto. base64 gana solo cuando el contenido es binario, porque ahí no hay alternativa: lo que no es texto no tiene «partes legibles». `

⚠️ Errores típicos

- **Poner en un PDF.** QP escapa bytes, pero un PDF comprimido no es texto: se destruye.
- **Poner en un texto con tildes.** Funciona, pero es ilegible y pesa más.
- **Olvidar el .** Sin esa cabecera, muchos clientes anciens ignoran todas las demás cabeceras MIME y muestran el PDF crudo.
- **Usar cuando querías texto plano y HTML.** Eso es `multipart/ alternativemixed` el cliente los muestra todos.

2.3 Ejercicio 3 — contra

Enunciado. Ana manda el mismo correo en texto plano y en HTML. Después lo manda otra vez con un Excel adjunto. ¿Qué va en cada caso?

` CASO A — texto plano + HTML: Content-Type: multipart/alternative; boundary="ALT1" ... --ALT1 Content-Type: text/plain; charset=UTF-8 (el texto) --ALT1 Content-Type: text/html; charset=UTF-8 (el

HTML) --ALT1--

El cliente ELIGE UNO. El último es el más Preferred, así que un cliente moderno muestra el HTML y el otro muestra el texto.

CASO B — texto + Excel adjunto: Content-Type: multipart/mixed; boundary="MIX1" ... --MIX1 Content-Type: text/plain; charset=UTF-8 (el texto) --MIX1 Content-Type: application/vnd.ms-excel; name="datos.xls" Content-Transfer-Encoding: base64 Content-Disposition: attachment; filename="datos.xls" (el Excel en base64) --MIX1--

El cliente los ve TODOS: muestra el texto y deja el Excel adjunto. `

Y se pueden anidar, que es lo que hacen los clientes reales:

` Content-Type: multipart/mixed; boundary="MIX1"

--MIX1 Content-Type: multipart/alternative; boundary="ALT1"

--ALT1 Content-Type: text/plain; charset=UTF-8 --ALT1 Content-Type: text/html; charset=UTF-8 --ALT1--

--MIX1 Content-Type: image/png; name="grafico.png" Content-Transfer-Encoding: base64 Content-Disposition: inline; filename="grafico.png" --MIX1-- `

`mixed → "hay varias cosas: elegí CUALES te Mostro" ├── alternative → "acá hay varias VERSIONES: elegí UNA" └── image/png → "esto va adentro, en línea"`

> ⚠️ **La palabra clave es `Content-Type`, no `Content-Transfer-Encoding`:** *alternativa* significa que hay versiones del **mismo** contenido y el cliente elige; *mixto* significa que hay contenidos **distintos** y el cliente los ve todos.

2.4 Ejercicio 4 — POP3 e IMAP en un escenario concreto

Datos. Ana tiene 4 200 mensajes en el servidor, repartidos en dos cuentas, , `nuevo@lugar.com` y `Viejo@lugar.com`. Tiene una notebook y un celular, y los dos se conectan todos los días.

a) ¿Qué pasa si configura POP3 en los dos, con «borrar del servidor al descargar»?

` Lunes, en la notebook: POP3 baja los 4 200 mensajes (4 200 × ~4 RTT... en realidad uno por mensaje: una sesión, pero un RETR por mensaje) y los BORRA del servidor.

Lunes, en el celular, una hora después: +OK maildrop has 0 messages → Ana no ve NADA.

Martes, en la notebook: +OK maildrop has 0 messages → Ana ya no tiene NADA en ningún lado, salvo lo que se descargó.

Peor: si borra en la notebook pero NO en el celular, entonces el celular sigue teniendo los 4 200 y la notebook no, y el problema es al revés. POP3 no se synchronize: cada cliente ve lo que le tocó. `

b) ¿Qué pasa si configura IMAP en los dos?

` Lunes, notebook: SELECT INBOX → 2 000 mensajes FETCH 1..50 → baja 50, NO los borra, marca \Seen en el servidor

Lunes, celular (una hora después): SELECT INBOX → 2 000 mensajes (los mismos) Lo que leyó en la notebook aparece con \Seen: no lo vuelve a mostrar como nuevo. Y las carpetas están todas: Trabajo, Familia, Archivo.

El archivo NO se duplica en los dos lados: es el mismo, en el servidor, y los dos clientes leen la misma copia. `

c) El costo de cada uno, en números.

` CON POP3 (borrando del servidor): 1er dispositivo : baja 4 200 mensajes. Supongamos 40 KB cada uno: 4 200 × 40 KB = 168 MB de tráfico y de disco. 2do dispositivo : 0 mensajes.

CON IMAP: 1er dispositivo : baja solo los que no había visto: 200 mensajes, 8 MB. 2do dispositivo : los mismos 200 que faltan, 8 MB. El servidor : los 4 200 mensajes siguen ahí, occupying disco (que es lo que el administrador tiene que pagar).

O sea: POP3 ahorra espacio en el servidor y paga en cada cliente; IMAP paga en el servidor y ahorra en cada cliente. `

d) ¿Y si Ana tiene un solo dispositivo y una línea lenta?

`POP3 gana, y por mucho: con IMAP tendría que dejar los 4 200 en el servidor y bajar 200; con POP3 baja una vez y los borra, y listo. La elección depende de la relación entre "número de dispositivos" y "espacio en el servidor", no de cuál protocolo es "mejor".`

2.5 Ejercicio 5 — *Dot-stuffing* con un mensaje que empieza con

Datos. Ana manda este mensaje, que tiene una línea con **dos puntos** al principio (inicio de línea = después de un):

` Hola Pedro:

..esto NO es el fin del mensaje, es texto .y esto tampoco

Saludos `

a) ¿Qué travels por el cable?

` REGLA DEL DOT-STUFFING: si una línea del mensaje empieza con un punto, se le AGREGA un punto más al principio. Solo si empieza con punto.

original en el cable ——————————————————————— ———————————————————————
Hola Pedro: Hola Pedro: (línea vacía) ..esto NO es el fin... ...esto NO es el fin... .y esto tampoco ..y esto tampoco (línea vacía) Saludos Saludos (fin) . ← el delimitador

OJO con la cuenta: "..esto" → "...esto" (2 puntos → 3) ".y esto" → "..y esto" (1 punto → 2) El delimitador final es UN punto solo, porque no es contenido. `

b) La transcripción SMTP completa, con el escapado:

`C: DATA S: 354 End data with <CR><LF>.<CR><LF> C: Subject: Prueba de dot-stuffing C: C: Hola Pedro: C: C: ...esto NO es el fin del mensaje, es texto C: ..y esto tampoco C: C: Saludos C: . S: 250 2.0.0 Ok: queued as 44C1F0AA1177`

c) ¿Qué recibe Pedro?

` Pedro, al quitar el stuffing (saca un punto de cada línea que empiece con punto), reconstruct este mensaje:

Hola Pedro:

..esto NO es el fin del mensaje, es texto .y esto tampoco

Saludos

IDÉNTICO al original. Ana no se dio cuenta de que estaba usando dos puntos, y no hizo falta que se diera cuenta. `

d) ¿Por qué el punto es el carácter de relleno natural?

Esta es la pregunta de examen, y la respuesta tiene tres partes:

` 1. POR QUÉ NO PUEDE SER OTRO CARÁCTER Because un protocolo de texto necesita un delimitador que sea un carácter que (a) aparezca en todas las versiones de todos los sistemas y (b) tenga un significado que lo haga identificable. El punto cumple las dos: es ASCII de 7 bits, está en todas las tablas de caracteres y en la línea de comandos representa «current directory» y «fin de frase».

2. POR QUÉ ES EL QUE YA ESTÁ EN USO Because SMTP heredó de telnet y de los protocolos de teletipo la convención de "el fin del texto es un punto solo". Reutilizar un carácter que ya hacía ese papel evita inventar un segundo marcador.
3. POR QUÉ EL ESCAPADO ES SOLO DE UN PUNTO Because si el delimitador fuera ".", entonces para que el receptor no lo confunda con el final, basta con que CUALQUIER línea que empiece con "." empiece con dos. Es la solución mínima: duplicar el carácter.

En cambio, si el delimitador fuera una cadena larga (como el "--boundary--" de MIME), el escapado tendría que ser más complejo.

Y el costo, que es lo que el protocolo paga: el receptor tiene que HACER algo en cada línea. Es trabajo de CPU del servidor, que es exactamente la razón por la que hoy los servidores grandes rechazan conexiones SMTP que hacen esto y aplican rate limiting. `

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2010-03, Pregunta 2 (8 puntos) — la arquitectura del correo

> **Fuente:** Examen / — 24 de febrero de 2010. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1003.pdf

Enunciado (texto literal del examen)

> **Pregunta 2 (8 puntos)**
>
> **a)** Dibuje un diagrama que describa la arquitectura de la aplicación correo electrónico. En el diagrama debe indicar cuáles son los roles de los sistemas que participan y los protocolos de aplicación que conozca que se pueden utilizar para intercambiar información entre ellos.
>
> **b)** Describa la serie de pasos que se aplican sobre un correo electrónico desde que el emisor lo escribe hasta que el destinatario lo lee, asumiendo que el emisor y el destinatario tienen casillas de correo de servidores distintos.

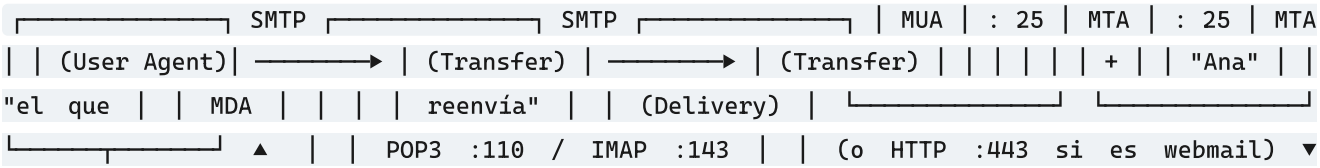
Solución oficial (del PDF)

Del apartado **a)** el PDF **publica un diagrama como imagen** y no un texto: en el extraído aparecen literalmente las palabras «Solución parte a)» y «Solución parte b)» sin contenido entre ambas. Del apartado **b)** la solución textual es:

> *El emisor escribe el mensaje de correo electrónico. El user agent del emisor envía por SMTP el correo a su mail server. El mail server del emisor envía por SMTP el correo al mail server del destinatario. El mail server del destinatario almacena el correo en la casilla. Más tarde, el destinatario se conecta a su mail server. Si utiliza POP3 o IMAP descarga el correo a su máquina. Si utiliza un webmail lo puede leer en el browser (descarga el correo por HTTP).*

(el diagrama del apartado a) es solución propia, reconstruido con los roles y los protocolos que la cátedraisease en la clase 7 de capa de aplicación)

a) El diagrama, con los roles rotulados



```
└──────────────────────────────────────────┐  ┌──────────┐  ┌──────────┐
                                           │  │  Buzón   │  │ "Pedro"  │
                                           └──└──────────┘  └──────────┘
```

Los rótulos que hay que poner en el diagrama, uno por actor:

Actor	Rótulo obligatorio	Puerto y protocolo hacia el siguiente
El programa de Ana	**MUA** — *Mail User Agent* (agente de usuario de correo)	SMTP al MTA propio, **25**
El servidor de Ana	**MTA** — *Mail Transfer Agent* (agente de transferencia)	SMTP al MTA remoto, **25**
El servidor de Pedro	**MTA** + **MDA** — *Mail Delivery Agent* (agente de entrega)	POP3 **110** o IMAP **143**
El programa de Pedro	**MUA**	—

Lo que hay que dibujar además, y es lo que más suma puntos: el **camino de ida y de vuelta** del correo son **protocolos distintos**. La **ida es SMTP, que es *push*** (el servidor empuja el mensaje). La **vuelta es POP3 o IMAP, que son *pull*** (el cliente tira del mensaje). **SMTP nunca se usa para leer el correo.** Ese es el error clásico.

También: la posibilidad de **webmail**, que agrega una cuarta flecha por **HTTP/HTTPS** y **el mismo MDA** de antes. La oficial lo menciona en b), y está bien mencionado.

b) Los cinco pasos — resolución con la solución oficial a la vista

La oficial da cinco pasos. Es **correcta y completa** para los 8 puntos. Lo que hay que agregar es la **precisión de los roles** y los **porqués**, que es lo que suma:

` PASO 1 — Ana escribe el mensaje en su MUA. Es un texto con cabeceras: From:, To:, Subject:, Date:.

PASO 2 — El MUA de Ana lo ENTREGA a su propio servidor de correo por SMTP (puerto 25, o mejor 587 con autenticación). ★ El MUA no habla con el servidor de Pedro: habla con el suyo. El correo todavía no salió de la organización.

PASO 3 — El MTA de Ana ENVÍA el mensaje por SMTP al MTA de Pedro. ★ Acá puede haber VARIAS etapas, porque el correo puede viajar por reenviadores. Y puede haber reintentos: si un servidor da 4xx, el mensaje queda en una COLA y se reintenta más tarde. Si da 5xx, se descarta y se manda un reporte de-states al emisor.

PASO 4 — El MTA de Pedro hace de MDA: ENTREGA el mensaje, es decir lo ESCRIBE EN EL BUZÓN de Pedro en el disco del servidor. ★ Nada de esto es extremo a extremo: el servidor de Pedro vio el contenido del mensaje. Es la diferencia con HTTPS.

PASO 5 — MÁS TARDE (y este es el detalle que hay que subrayar) Pedro conecta su MUA a su servidor y LEE: · con POP3 (110): el cliente baja los mensajes y, opcionalmente, los borra del servidor. El estado queda en el cliente. · con IMAP (143): el cliente los mira y deja todo en el servidor. El estado (carpetas, flags) queda en el servidor. · con un webmail: su MUA es un navegador, y lee por HTTPS. El protocolo de aplicación es HTTP, no POP3 ni IMAP. `

Respuesta redactada, lista para copiar:

> **a)** La arquitectura del correo tiene **tres roles**: el **MUA** (*Mail User Agent*), que es el programa con el que el usuario escribe y lee; el **MTA** (*Mail Transfer Agent*), que es el servidor de correo que **reenvía** los mensajes entre organizaciones; y el **MDA** (*Mail Delivery Agent*), que es el que **deposita** el mensaje en el buzón del destinatario y que típicamente corre en el mismo proceso que el MTA receptor. Los protocolos son: **SMTP** (puerto **25**) entre el MUA y su MTA, y entre un MTA y otro MTA —que es un protocolo **push**—; **POP3** (puerto **110**), **IMAP** (puerto **143**) o **HTTP/HTTPS** si es webmail, entre el MUA del receptor y su servidor —que son protocolos **pull**—; y **DNS** (tipo) para descubrir a qué servidor hay que entregarle el correo de cada dominio.
>
> **b)** Los pasos son: **1)** el emisor escribe el mensaje en su **MUA**; **2)** el MUA lo entrega por **SMTP** a su propio servidor de correo; **3)** el **MTA** del emisor envía el mensaje por **SMTP** al **MTA** del destinatario, que puede estar a varias etapas de distancia y donde puede haber reintentos por errores temporales (); **4)** el MTA del destinatario actúa como **MDA** y **deposita el mensaje en el buzón** del destinatario en el servidor; **5)** más tarde el destinatario conecta su MUA a su servidor y lee: con **POP3** descarga los mensajes al cliente y opcionalmente los borra del servidor; con **IMAP** los consulta y el estado queda en el servidor; y con un **webmail** los lee en el navegador descargándolos por **HTTP/HTTPS**.

Análisis de la solución oficial.

✅ **Lo que está bien:** los cinco pasos están en el orden correcto, nombra al *user agent*, a los dos *mail server*, y menciona las tres formas de leer (POP3, IMAP, webmail). Para los 8 puntos de conceptual es suficiente.

⚠️ **Lo que la oficial NO dice y suma puntos:**

1. **No nombra los roles con sus siglas.** El enunciado del apartado a) pide **explícitamente** «cuáles son los roles». Saying «el user agent» y «el mail server» sin escribir **MUA**, **MTA** y **MDA** deja puntos en la mesa: la cátedra espera las tres siglas, con su glosa en inglés.
2. **No dice que SMTP es *push* y POP3/IMAP *pull*.** Es la distinción de diseño de toda la clase, y es lo que permite entender por qué son dos protocolos distintos.
3. **No menciona el reintento ni la cola.** Un significa «vuelvo a intentar», y eso explica por qué un correo puede tardar minutos u horas y llegar igual. **Y un genera el *bounce***, que es un correo de retorno que le avisa al emisor que su mensaje no llegó.
4. **No dice que el MDA se ejecuta en el servidor del destinatario.** Muchos alumnos dibujan un cuarto servidor que no existe.

> **Nota explícita contra la oficial:** la oficial es **correcta** para el apartado b) y su redacción es la que se corrige. Mi lectura difiere en un punto menor: la oficial trata «el mail server del destinatario» como un solo actor, y eso es correcto **porque el MTA y el MDA son el mismo proceso**. Pero si el enunciado pide **los roles**, hay que **separarlos en el diagrama aunque compartan máquina**, porque son responsabilidades distintas —uno decide a dónde va, el otro escribe el archivo en el buzón— y en una red real pueden estar separados (el *MDA* suele ser o un antivirus, no el).

3.2 ⭐ Examen 2008-12, Pregunta 4 (7 puntos) — el de POP3

> **Fuente:** Examen / — 20 de diciembre de 2008. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0812.pdf

Enunciado (texto literal del examen)

> **Pregunta 4 (7 puntos)**
> El protocolo POP3 transmite información de control por el mismo canal de datos. ¿Qué mecanismo utiliza para evitar que la información de señalización se confunda con los cuerpos de los mensajes? Justifique su respuesta.

Solución oficial (del PDF, textual)

> El protocolo POP3 se basa en el intercambio de mensajes de control (orientados a texto) y transferencias de datos (con contenido arbitrario). El protocolo reserva la secuencia para indicar el fin de mensaje. En caso que el mensaje contenga la misma secuencia como parte del mensaje se realiza el *stuffing* de la secuencia convirtiéndola en .

La resolución completa, con la oficial a la vista

Paso 1 — El problema, que es lo que el enunciado plantea en la primera frase.

` POP3 transmite CONTROL y DATOS por el MISMO canal TCP.

Control = "USER ana", "+OK", "RETR 1", "-ERR no such message" (líneas cortas) Datos = el mensaje entero, que es ARBITRARIO

Si los dos van por el mismo canal, el receptor tiene que poder distinguir "esto es un comando" de "esto es el contenido de un mensaje". `

Paso 2 — La respuesta oficial: se reserva una secuencia que no puede aparecer.

` POP3 reserva la secuencia

. (CRLF.CRLF en la notación de la oficial)

y la usa como el ÚNICO terminador de un mensaje multi-línea.

Es la misma idea que un "delimitador reservado" en cualquier formato de texto. `

Paso 3 — Pero acá está la trampa, y es lo que el enunciado insinúa.

` ⚠️ EL PROBLEMA REAL:

POP3 transfiere "contenido arbitrario" (esas son las palabras de la oficial). Y un mensaje de correo es un archivo de TEXTO que puede contener CUALQUIER cosa... incluido un renglón que sea solo un punto.

Si un mensaje contiene: Hola Pedro: . Saludos

...el POP3, al recibirlo, lee "." y piensa: "SE TERMINÓ EL MENSAJE". Lo que viene después ("Saludos") lo interpreta como una línea de CONTROL, es decir, como un comando que no existe, y la sesión se rompe.

O sea: el mecanismo de la oficial RESUELVE el problema de distinguir control de datos, pero NO RESUELVE el problema de que el terminador aparezca dentro de los datos. Para eso está el segundo paso de la oficial. `

Paso 4 — La respuesta oficial: el *dot-stuffing* (relleno de punto).

` La oficial: "En caso que el mensaje contenga la misma secuencia como parte del mensaje se realiza el stuffing de la secuencia convirtiéndola en CRLF..CRLF".

Es decir: ANTES de transmitir, el emisor busca toda línea que empiece con un punto y le AGREGA un punto más. Después, el receptor, al recibir, saca un punto de cada línea que empiece con punto.

original en el cable ——————————————— ——————————————— Hola Pedro: Hola Pedro: . (vacía) . .. ← era "." y se volvió ".." Saludos Saludos . . ← el terminador de verdad

Con esto, un "." suelto ya NUNCA puede producir ".", porque en el cable siempre viaja como "..". El terminador queda RESERVADO de verdad. `

Paso 5 — El ejemplo completo, que es lo que hay que saber escribir.

` Mensaje original de Ana (tiene un renglón que es un punto solo):

Hola Pedro: . Saludos Ana

Lo que Ana MANDA por POP3 con RETR (con stuffing):

Hola Pedro: .. Saludos Ana . ↑ el punto de cierre de la transacción, que no es del mensaje

Lo que POP3 le ENTREGA al cliente (deshaciendo el stuffing):

Hola Pedro: . Saludos Ana `

Respuesta redactada, lista para copiar:

> **a)** POP3 transmite la **información de control** (los comandos y las respuestas y) y la **información de datos** (el mensaje de correo) por el **mismo canal TCP**, que es texto orientado a líneas. Para que el receptor no confunda una cosa con la otra, **reserva la secuencia como único terminador de un mensaje multi-línea**: un renglón que sea un punto aislado marca el final del mensaje, y todo lo que venga después se interpreta como control.
>
> **b)** Justificación: como el contenido que se transfiere es **arbitrario**, un mensaje puede contener un renglón que sea un punto solo, y entonces la secuencia reservada aparecería **dentro de los datos** y el receptor cortaría el mensaje antes de tiempo, QPQreat interpretando el resto como comandos. Por eso, además de reservar la secuencia, el protocolo aplica **dot-stuffing**: el emisor **duplica el punto** al principio de toda línea que empiece con un punto (convirtiendo en), y el receptor **elimina ese punto de relleno** al recibir. Así, en el cable, **nunca** puede aparecer un que sea contenido: esa secuencia queda **reservada exclusivamente** para el fin de mensaje.

Análisis de la solución oficial

✅ **Lo que la oficial acierta:** identifica correctamente el problema (mismo canal para control y datos), la solución (secuencia reservada) y el segundo paso (dot-stuffing convirtiendo en). Los 7 puntos están ahí.

⚠️ **Lo que la oficial no dice y hay que agregar para el garantiza los 7 puntos:**

1. **No justifica POR QUÉ el punto es un buen carácter de relleno**, y esa es media pregunta. La respuesta: es un carácter **ASCII de 7 bits presente en todas las tablas**, tiene un **significado semántico** reconocible («fin de frase»), y **ya venía usándose** como terminador en los protocolos de teletipo. Agregar que el escapado es de **un solo carácter** (duplicar) es lo que hace que la solución sea mínima.
2. **No aclara la diferencia entre el terminador de POP3 y el de SMTP.** Los dos usan «un punto solo al principio de una línea», pero en **SMTP** el stuffing ocurre en la fase (el cliente hacia el servidor) y en **POP3** ocurre en la fase de transferencia (el servidor hacia el cliente). **La misma idea, dos protocols.**
3. **No nombra el término inglés**: *dot-stuffing*, y su primo en capa 2 es el *byte-stuffing* (el de PPP, que escapa con un). Ver §1.1 del Examen P3, que pregunta exactamente eso para PPP. **El mismo mecanismo, dos capas.**

> **Nota contra la oficial:** no hay diferencia conceptual, pero la oficial **redacta el stuffing como si se transformara la secuencia completa**. Lo que ocurre no es que se convierta en como una unidad, sino que **toda línea que empiece con un punto recibe un punto extra al principio**. La diferencia se nota en un mensaje que empieza con (Ejercicio 5): ahí el de la oficial se volvería (tres puntos), mientras que la regla real da por una línea que ya tenía dos puntos. **En la práctica dan lo mismo, pero la regla de la línea que empieza con punto es la que hay que saber.**

3.3 Problema relacionado — el Examen 2004-12, Ejercicio 2 (el diálogo SMTP)

> **Fuente:** Examen — 21 de diciembre de 2004. **Sin solución oficial publicada** en el material del curso; se resuelve abajo. *(solución propia)*

Enunciado (texto del examen). Se transcribe una sesión SMTP real en la que las líneas son del servidor y las del cliente:

```
S: 220 mta4.in.adinet.com.uy ESMTP Service (7.0.027) ready C: mail from: <origen@dominio.com.uy> S: 250 MAIL FROM:< origen@dominio.com.uy > OK C: rcpt to: <dest1@dominio.com.uy > S: 250 RCPT TO:<dest1@dominio.com.uy> OK C: rcpt to: <dest2@dominio.com.uy> S: 550 RCPT TO:<dest2@dominio.com.uy> User unknown C: data S: 354 Start mail input; end with <CRLF>.<CRLF> C: Subject: Hola a todos C: C: texto del correo..... C: . S: 250 <40AB06AF005B39F9> Mail accepted C: quit S: 221 mta4.in.adinet.com.uy QUIT
```

a) ¿Qué diferencia hay entre y ?

` MAIL FROM = el REMITENTE. Solo uno por mensaje. Va primero, porque el servidor necesita saber a quién avisarle si algo falla. RCPT TO = un DESTINATARIO. Uno o varios, y se puede repetir.

En la transcripción: un MAIL FROM, dos RCPT TO, y el segundo falla. `

b) ¿Qué pasó con ?

` 550 = error PERMANENTE: el usuario no existe. El mensaje NO se reintenta. Para ese destinatario en particular, el envío terminó ahí. Y como el 550 es permanente, el MTA va a generar un reporte de-states (bounce) para avisarle al emisor que su correo a dest2 no llegó.

Con un 4xx en lugar de 550 (por ejemplo 450 mailbox busy), el mensaje quedaría en una cola y se reintentaría más tarde: el envío NO se abandona. `

c) ¿Qué línea del mensaje está escapada, y por qué?

` El original decía algo como: .---------- o un renglón que empezaba con un punto

En el cable aparece "texto del correo....." con diez puntos pegados: el último de esos es un punto final de línea escapado, porque la línea del mensaje empezaba con un punto. Si no se hubiera escapado, el POP3/SMTPreceiver lo habría tomado como el terminador del mensaje. `

d) ¿Qué utilidad tiene enumerar usuarios válidos? *(lo que pregunta el apartado a) del enunciado)*

` ★ La utilidad legítima es la DEPURACIÓN: un administrador necesita saber si una lista de direcciones que tiene en un archivo todavía existe. El problema de esa dirección, el bounce por los 550, sirve justamente para eso.

★ La utilidad ABUSIVA es el enumerado de usuarios (el «harvesting»), que es una de las formas más viejas de spam: se prueban millones de direcciones en un dominio y se guarda las que no dan 550, para después mandarles publicidad.

Y por qué el 550 es unaRespuesta para el atacante: distingue un error permanente de uno temporal, así que no se equivoca nunca.

★ La defensa es la del apartado b) del ejercicio: contar los 550 que generan por IP de origen y bloquear esa IP cuando pasa un umbral. `

Este problema es el complemento perfecto del Examen : los dos son sobre correo, y los dos aparecen juntos en la historia de la cátedra. Uno pregunta la arquitectura y los roles; el otro muestra una sesión real y pide qué hacer con la información que se filtra por ella.

PARTE 4 — CIERRE

📌 Resumen

1. **Tres roles**: **MUA** (escribe y lee), **MTA** (reenvía) y **MDA** (deposita en el buzón). El MTA y el MDA suelen ser el mismo proceso.
2. **El correo es *store-and-forward***: es el único protocolo de aplicación que **no** es extremo a extremo, y por eso un servidor intermedio puede leer el mensaje.
3. **SMTP es *push***, puerto **25** (servidor a servidor), **587** (usuario a servidor, con) y **465** (TLS implícito). **POP3 e IMAP son *pull***: **110/995** y **143/993**.
4. **El diálogo SMTP**: → (uno) → (uno o varios, y puede fallar sin tumbar el envío) → → el mensaje → → .
5. **Los códigos son de tres dígitos**: éxito, error temporal (cola y reintento), error permanente (bounce).
6. **MIME existe porque RFC 822 era ASCII de 7 bits**: los binarios podían traer y romper el transporte. dice **qué** es; dice **cómo viaja**.
7. ** / / **: QP para** texto **(legible, ~+20 %), base64 para** binario **(ilegible,** +33 %**).
8. ***** = ***versiones del mismo mensaje, el cliente*** *elige una**. ***** = ***contenidos distintos, el cliente los ve*** ***todos**.
9. **POP3** tiene **dos estados** y su `DELE` **solo marca**: borra de verdad en el `QUIT`. Sus comandos van **en claro**, y por eso el puerto seguro es **otro puerto (995)**.
10. **POP3 e IMAP se diferencian en dónde vive el estado**: en el **cliente** o en el **servidor**. Todo lo demás (carpetas, búsqueda, sincronización) sale de ahí.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. ¿Cuáles son los tres roles del correo y qué protocolo usa cada uno hacia el siguiente?

▶ Respuesta

2. ¿Por qué SMTP y POP3 no son el mismo protocolo, si los dos mueven correo?

▶ Respuesta

3. ¿Qué diferencia hay entre un y un en un ? ¿Y qué hace el MTA con cada uno?

▶ Respuesta

4. ¿Por qué existe MIME y qué problema concreto resuelve?

▶ Respuesta

5. ¿Cuándo va y cuándo va ? ¿Por qué no al revés?

▶ Respuesta

6. Explicá qué diferencia hay entre `POP3` e `IMAP`.

▶ Respuesta

7. ¿Por qué los comandos de POP3 van en claro, y qué se hace al respecto?

▶ Respuesta

8. Un mensaje contiene una línea que es un punto solo, en medio del cuerpo. ¿Qué pasa?

▶ Respuesta

🔗 Referencias

- **Examen / ** — Pregunta 2 (8 pts): ★ arquitectura del correo, los tres roles, los protocolos, y los cinco pasos del envío (el apartado a) está publicado como imagen)
- *Examen / ** — Pregunta 4 (7 pts): ★ el de POP3 y el* dot-stuffing*
- *Examen ** — Ejercicio 2: la transcripción real de una sesión SMTP con un y el* dot-stuffing*; el problema de enumerar usuarios válidos* (sin solución oficial publicada: solución propia)*
- *Examen ** — Problema 2: un* front-end* que reparte sesiones POP3 entre tres servidores (aplicación del mismo tema)
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 7 §7.1 («Correo electrónico») y cap. 6 para la comparación de throughput
- **RFC 5321** — *Simple Mail Transfer Protocol* (2008, *Internet Standard*; reemplaza a) — *verificar*
- **RFC 5322** — *Internet Message Format* (2008; reemplaza a) — *verificar*
- **RFC 1939** — *Post Office Protocol version 3* (1996) — *verificar*
- **RFC 3501** (2001) → **RFC 9051** (2021), *IMAP4rev2* — *verificar* el estado vigente
- **RFC 2045-2049** — MIME (1996) — *verificar*
- **RFC 4954** — SMTP (2007) — *verificar* · **RFC 3207** — (2001) — *verificar*
- **IANA**, registro de puertos (verificado en §15): **25** SMTP · **110/995** POP3/POP3S · **143/993** IMAP/IMAPS · **465** submissions · **587** submission
- **Clases anteriores:** 8, 9 (HTTP) · 10, 11, 12 (DNS) — **Clase siguiente:** 14 — Seguridad en la aplicación: TLS, autenticación y correo seguro